

# SevaSetu — Security and Access Document

## 1. Purpose

This document defines how SevaSetu should protect users, organizations, volunteer data, event information, Karma, attendance records, emergency operations, and AI Agent functionality.

The goal is not to make SevaSetu unnecessarily complicated.

The goal is to ensure that:

- Users can only access information they are allowed to see.
  - Volunteers cannot manipulate their own Karma or attendance.
  - NGOs cannot modify another NGO's data.
  - The AI Agent cannot make sensitive decisions by itself.
  - Core records remain trustworthy.
  - A failure in one part of the system does not corrupt another part.
  - Security rules are enforced by the backend/database rather than relying only on the frontend.
- 

## 2. Security Principles

SevaSetu should follow these principles throughout development.

### 2.1 Supabase is the source of truth

Supabase is the authoritative database for:

- Users
- Volunteer profiles
- NGO profiles
- Events
- Event requirements
- Registrations
- Rosters
- Assignments
- Attendance
- Karma transactions
- Reliability records
- Platform analytics

Firebase must not become a second master database for these records.

---

## 2.2 The Agent recommends; humans decide

The Event Intelligence Agent can:

- Analyze events
- Analyze previous event information
- Suggest preparation steps
- Suggest possible risks
- Provide guidance
- Help volunteers understand events

The Agent must NOT independently:

- Create an emergency
- Broadcast an emergency
- Finalize a roster
- Modify attendance
- Award Karma
- Change reliability
- Approve an NGO
- Delete authoritative records
- Override an NGO decision

---

## 2.3 Never trust the frontend

A button being hidden in the frontend is not security.

For example, even if the frontend hides:

"Delete Event"

from volunteers, the backend must still reject a volunteer who attempts to call the delete API directly.

Every sensitive action must be checked on the server/database.

---

## 2.4 Verified records are protected

The following records should be treated as authoritative:

- Attendance

- Completed assignments
- Karma transactions
- Reliability history
- Confirmed rosters
- Emergency actions
- Audit logs

Ordinary users must not be able to directly modify these records.

---

## 2.5 Minimum necessary access

Every user and service should receive only the permissions it actually needs.

For example:

The Agent should not receive unrestricted access to the entire Supabase database.

---

## 3. Recommended Authentication Method

### Clerk Authentication

SevaSetu should use **Clerk** as the primary authentication system.

This is appropriate because SevaSetu needs:

- User registration
- Login
- Logout
- Session management
- Password/security handling
- Role-aware access
- Protected application routes

The application should not build its own password authentication system.

Clerk should handle the identity.

SevaSetu should handle the application's permissions and business rules.

---

### 3.1 Authentication Flow

The basic flow should be:

User

↓

Clerk Login / Registration

↓

Authenticated Session

↓

Backend verifies identity

↓

Application finds user profile

↓

Role is determined

↓

Access is granted according to role

---

### 3.2 Authentication vs Authorization

These are different.

#### Authentication

"Who are you?"

Handled primarily by Clerk.

#### Authorization

"What are you allowed to do?"

Handled by:

- Backend permissions
  - Supabase Row-Level Security
  - Application role checks
  - Ownership checks
- 

### 4. User Roles

SevaSetu should have four security roles.

#### Role 1 — Volunteer

This includes both:

- NGO-affiliated volunteers

- Independent/freelance volunteers

An independent volunteer does not need to belong to an NGO to participate in public events.

**Volunteer CAN:**

- Create and edit their own profile
- Add skills
- Add availability
- Add interests
- Add experience
- Add certification information
- Browse public NGO events
- View public NGO transparency information
- Register for eligible events
- Cancel their own registration according to event rules
- View their own roster assignments
- View their own attendance history
- View their own service history
- View their own Karma
- View their own Karma transactions
- View their own reliability information where exposed
- Ask the Agent questions about events
- Receive event notifications
- Accept/decline emergency requests when eligible

**Volunteer CANNOT:**

- Modify another volunteer's profile
- Modify another volunteer's attendance
- Award themselves Karma
- Modify Karma transactions
- Modify their reliability score
- Create NGO events
- Modify NGO event requirements
- Finalize NGO rosters

- Mark themselves as officially attended
  - Create emergency broadcasts
  - Deploy emergency volunteers
  - Access private NGO information
  - Access another volunteer's private information
  - Access platform administration functions
- 

## **5. Role 2 — NGO Coordinator / NGO Account**

The NGO role represents authorized members of an NGO.

An NGO should be associated with an organization record.

### **NGO CAN:**

- Create and manage its own NGO profile
- Create events
- Edit its own events
- Define event requirements
- Define required skills
- Define required certifications
- Define shifts
- View registrations for its events
- Request volunteer matching
- Review matching recommendations
- Review match explanations
- Generate rosters
- Modify proposed rosters
- Confirm final rosters
- View attendance for its events
- Mark attendance through authorized workflows
- Complete its own events
- View its organization's analytics
- View its volunteer participation
- Initiate authorized emergency workflows

- View its transparency/activity information
- Use the Event Intelligence Agent for its own events

**NGO CANNOT:**

- Access another NGO's private event data
  - Modify another NGO's events
  - Modify another NGO's volunteers
  - Award arbitrary Karma
  - Directly edit Karma transactions
  - Directly edit reliability records
  - Override platform security
  - Access Clerk administrative credentials
  - Access Supabase service credentials
  - Make the Agent perform restricted actions
  - Mark unauthorized users as attended
  - Modify historical verified records without an auditable correction workflow
- 

## **6. Role 3 — Platform Administrator**

This is the internal SevaSetu administrative role.

It should be extremely limited.

**Admin CAN:**

- Manage platform users when necessary
- Investigate reports
- Monitor platform activity
- View system-level analytics
- Investigate security incidents
- Manage platform configuration
- Review audit logs
- Handle exceptional account issues
- Disable abusive accounts according to platform policy

**Admin CANNOT casually:**

- Change Karma without a reason

- Change attendance without an audit trail
- Modify NGO records without logging the action
- Modify volunteer records unnecessarily
- Access private data unrelated to an operational/security need

Every sensitive administrative action should be logged.

---

## 7. Role 4 — Public User

A public user is an unauthenticated visitor or a user who has not joined an NGO.

### Public users **CAN**:

- View public NGO profiles
- View public NGO transparency/activity indicators
- View public events
- View publicly available event information
- Discover community opportunities

### Public users **CANNOT**:

- View private volunteer information
  - View volunteer availability
  - View private NGO operational information
  - View private event coordination information
  - View attendance records
  - View Karma history
  - View reliability details
  - Register for restricted functionality without authentication
- 

## 8. The AI Agent Is NOT a User Role

The Agent should be treated as a **restricted system service**, not as a human user.

It should have its own service identity.

The Agent should have access only to the data required for the current task.

For example:

Agent asks:

"Analyze this event"

Backend provides:

- Event details
- Required skills
- Event location
- Event type
- Relevant previous event information

Agent returns:

- Preparation suggestions
- Risk considerations
- Recommendations

Backend remains authoritative.

The Agent should not receive unrestricted database credentials.

---

## **9. Supabase Row-Level Security**

Supabase Row-Level Security, or RLS, should act as the final database-level protection.

In simple terms:

Even if someone manages to send a request directly to the database API, the database itself should refuse data they are not allowed to access.

---

## **10. User Profile Rules**

### **Volunteers**

A volunteer can:

- Read their own private profile
- Create their own profile
- Update their own profile

A volunteer cannot:

- Read another volunteer's private profile
- Update another volunteer's profile

Public information should be stored or exposed separately from private information.

---

## **11. NGO Profile Rules**

### **NGO members**

An NGO member can:

- Read their organization's profile
- Update permitted organization fields

They cannot:

- Modify another NGO's profile

### **Public users**

Public users can only access fields intentionally marked as public.

Private operational information must remain protected.

---

## **12. Event Rules**

### **NGO event ownership**

An NGO can:

- Create events belonging to its organization
- Read its own events
- Update its own events
- Manage registrations for its own events

An NGO cannot:

- Read private operational information belonging to another NGO
- Modify another NGO's event

### **Public event access**

If an event is published publicly:

Public users may see:

- Event title
- Description
- Date
- Time
- Public location information

- Required skills
- NGO public profile

They should not automatically receive:

- Private volunteer lists
  - Internal notes
  - Private contact information
  - Internal operational details
- 

### **13. Registration Rules**

A volunteer can:

- Create their own registration
- View their own registration
- Cancel their own registration where allowed

An NGO can:

- View registrations for its own events

A volunteer cannot:

- Create registrations on behalf of another volunteer
- Modify another volunteer's registration

The database should prevent duplicate registrations.

---

### **14. Roster Rules**

The roster is more sensitive than a normal event.

#### **Before confirmation**

The NGO can:

- View proposed assignments
- Replace volunteers
- Review conflicts
- Review unfilled positions

#### **After confirmation**

The final roster should become protected.

Changes should require:

- Authorized NGO action
- Validation
- Audit logging

Volunteers cannot modify the final roster.

---

## **15. Attendance Rules**

Attendance is a verified record.

### **Volunteer**

Can:

- View their own attendance

Cannot:

- Create attendance
- Change attendance
- Delete attendance

### **NGO**

Authorized NGO staff can:

- Record attendance for their own event
- Correct attendance through an authorized correction workflow

Every correction should be logged.

---

## **16. Karma Security Rules**

Karma should be transaction-based.

For example:

Event completed

↓

Attendance verified

↓

Service contribution calculated

↓

Karma transaction created

↓

Volunteer Karma total updated/calculated

A volunteer must never be able to send:

POST /karma

```
{  
  "points": 5000  
}
```

and award themselves points.

Karma creation must only happen through trusted backend workflows.

---

## **17. Reliability Security Rules**

Reliability must be calculated from verified participation data.

Users cannot directly edit:

- Attendance history
- No-show records
- Completion records
- Reliability score

Karma and Reliability must remain separate.

Karma

= Contribution

Reliability

= Consistency

Karma should not directly become the matching score.

---

## **18. Transparency Security**

NGO transparency information should distinguish between:

### **Platform-derived information**

Example:

"24 events completed"

Source:

SevaSetu event records.

## **NGO-provided information**

Example:

"Funds utilized for community programs"

Source:

Information submitted by NGO.

The platform should never imply that an NGO-provided claim has been independently verified unless it actually has been verified.

SevaSetu should not automatically label an NGO:

- Fraudulent
- Fake
- Legitimate
- Certified

based solely on platform indicators.

---

## **19. Firebase Security**

Firebase is intended for Agent-specific context/state and real-time or temporary workflows.

It should NOT become a duplicate copy of:

- Volunteers
- NGO profiles
- Attendance
- Karma
- Reliability
- Event master records

Firebase data should have its own access rules.

The Agent should only access the Firebase records necessary for its operation.

---

## **20. Agent Security**

The Agent should operate through a controlled backend interface.

Preferred flow:

User

↓

Backend

↓

Permission Check

↓

Relevant Data Selected

↓

Agent

↓

Recommendation

↓

Backend

↓

User

Not:

User

↓

Agent

↓

Entire Database

The second architecture should be avoided.

---

## 21. Emergency Security

Emergency functionality requires additional protection.

The correct workflow is:

Authorized NGO

↓

Create Emergency Request

↓

Backend Authorization

↓

Identify Eligible Volunteers

↓

Send Alert

↓

Volunteer Accepts / Declines

↓

Live Confirmation Count

↓

Backup Volunteers if Needed

↓

Human Deployment Decision

The Agent must not be able to initiate the emergency.

The Agent can provide preparation or risk recommendations.

---

## 22. Error Handling Philosophy

Every error should answer three questions:

### 1. What happened?

Explain the problem in simple language.

### 2. What can the user do?

Provide a useful next action.

### 3. What should the system do?

Log the technical details privately without exposing them to the user.

For example:

Bad:

Error 500: foreign key violation.

Better:

We couldn't register you for this event. The event may no longer be available. Please refresh and try again.

The developer logs should contain the technical error.

The user should receive the safe explanation.

---

## 23. Authentication Errors

### **Invalid login**

User sees:

We couldn't sign you in. Please check your credentials and try again.

Do not reveal whether a specific email exists.

---

### **Expired session**

User sees:

Your session has expired. Please sign in again.

The system should redirect to authentication.

---

### **Unauthorized request**

User sees:

You don't have permission to perform this action.

Do not expose internal authorization rules.

---

## **24. Profile Errors**

### **Duplicate profile**

If a user already has a profile:

Your profile already exists.

Do not create another profile accidentally.

---

### **Invalid information**

Example:

Invalid availability or malformed location.

Show:

Please check the highlighted information and try again.

Do not save partially invalid data.

---

## **25. Event Errors**

### **Event creation failure**

User sees:

We couldn't create this event. Please check the event details and try again.

The incomplete event should not appear as a valid published event.

---

### **Event already full**

User sees:

This event has reached its volunteer capacity.

Do not allow the frontend alone to enforce capacity. The backend/database must also enforce it.

---

### **Event cancelled**

If a volunteer attempts to register:

This event is no longer accepting registrations.

---

## **26. Matching Errors**

If matching fails:

We couldn't generate recommendations right now. Please try again.

The system should never silently return misleading matches.

If there are genuinely no eligible volunteers:

No eligible volunteers were found for the current requirements.

This is different from a technical failure.

---

## **27. Scheduling Errors**

If scheduling cannot fill all positions:

The roster could not be fully filled. 3 positions remain unassigned.

Do not pretend that an incomplete roster is successful.

---

### **Scheduling conflict**

Show:

This volunteer is already assigned to another overlapping shift.

The system should prevent the conflict rather than merely warning about it.

---

## **28. Attendance Errors**

### **Unauthorized attendance modification**

Return:

You are not authorized to modify this attendance record.

---

### **Duplicate attendance**

The system should prevent duplicate attendance records.

---

### **Event already closed**

Show:

Attendance for this event is already finalized.

If corrections are allowed, use an explicit correction process with an audit trail.

---

## **29. Karma Errors**

### **Karma generation failure**

Do not mark the event as successfully processed if Karma creation fails halfway through.

Use a safe transaction/retry process.

User-facing message:

Your event participation was recorded, but Karma is still being processed.

The system should retry without creating duplicate Karma.

---

## **30. Agent Errors**

The Agent should be treated as an optional intelligence layer.

If the Agent is unavailable:

The Event Intelligence Assistant is temporarily unavailable. Your event data and core platform features are still working.

The platform must continue operating without the Agent.

Never allow an Agent outage to:

- Delete data
- Break attendance
- Prevent event browsing

- Corrupt rosters
- Change Karma

---

### 31. Agent Hallucination Protection

Agent responses must not be treated as authoritative facts unless they originate from trusted platform data.

For example, if the event requires:

5 volunteers

2 first-aid trained volunteers

the Agent must not invent:

8 volunteers

4 doctors

The Agent should distinguish:

#### **Known event information**

from

#### **General recommendations**

---

### 32. Firebase Failure

If Firebase becomes unavailable:

Core Supabase functionality should continue where possible.

The system should degrade gracefully.

For example:

Core platform

↓

Still works

Agent context

↓

Temporarily unavailable

Do not allow temporary Firebase failure to corrupt Supabase records.

---

### 33. Supabase Failure

If Supabase is unavailable:

The system should not pretend that database operations succeeded.

For example:

Do not show:

Registration successful

until the registration has actually been saved.

Show:

We couldn't complete your registration right now. Please try again.

---

### 34. Notification Failure

If an email or notification fails:

The underlying platform operation should not necessarily fail.

Example:

Registration saved

↓

Email failed

The registration should remain successful.

The notification system should retry separately.

---

### 35. Maps API Failure

If the Maps API fails:

The platform should not crash.

Possible fallback:

- Display event location without distance ranking
- Use stored location data
- Mark distance as temporarily unavailable

The matching system should clearly indicate when proximity information could not be calculated.

---

### 36. Security Edge Cases Before Launch

The following cases should be explicitly tested.

### **Account & Authentication**

- User registers twice.
  - User logs in from multiple devices.
  - Session expires during an operation.
  - User logs out and uses the browser's back button.
  - User attempts to access another user's URL directly.
  - User changes role-related information from the frontend.
  - Disabled user attempts to log in.
  - Deleted user attempts to reuse an old session.
- 

### **Volunteer Edge Cases**

- Volunteer has no skills.
  - Volunteer has no availability.
  - Volunteer has no experience.
  - Volunteer has no reliability history.
  - Volunteer has no Karma.
  - Volunteer has a very high Karma value.
  - Volunteer tries to register twice.
  - Volunteer tries to register after capacity is reached.
  - Volunteer tries to register after the event closes.
  - Volunteer tries to register for an event with incompatible mandatory certification.
  - Independent volunteer participates without NGO affiliation.
- 

### **Event Edge Cases**

- Event has zero required volunteers.
- Event capacity is smaller than the number of mandatory roles.
- Event date is in the past.
- Event registration deadline has passed.
- Event is cancelled after volunteers register.
- Event requirements are changed after matching.

- Event location is missing.
  - Event has overlapping shifts.
  - Event has more registrations than capacity.
  - Event is deleted while registrations exist.
- 

### **Matching Edge Cases**

- No volunteers match.
- Only one volunteer matches.
- Several volunteers have identical scores.
- Volunteer has required skills but is unavailable.
- Volunteer is nearby but lacks required certification.
- Volunteer has high reliability but poor skill compatibility.
- Maps API is unavailable.
- Volunteer location is missing.
- Event location is missing.
- Matching weights are changed.

The matching engine must always produce deterministic results for identical inputs.

---

### **Scheduling Edge Cases**

- Two shifts overlap.
  - Volunteer is assigned twice.
  - Volunteer becomes unavailable after roster generation.
  - Event capacity changes after roster generation.
  - A matched volunteer cancels.
  - There are not enough qualified volunteers.
  - NGO manually replaces a volunteer.
  - NGO attempts to confirm an invalid roster.
- 

### **Attendance Edge Cases**

- Volunteer never attends.
- Volunteer attends but is not on the roster.

- Attendance is recorded twice.
  - NGO accidentally marks the wrong volunteer.
  - Attendance is corrected after event completion.
  - Event is completed before attendance is finalized.
- 

### **Karma Edge Cases**

- Same event attempts to award Karma twice.
- Attendance is reversed after Karma was awarded.
- Event is cancelled after Karma processing.
- Volunteer tries to modify Karma.
- Two backend processes attempt to create the same Karma transaction.
- Karma calculation fails halfway through.

Every Karma transaction should have a unique reference to prevent duplicate awards.

---

### **Reliability Edge Cases**

- New volunteer has no history.
- Volunteer has only one event.
- Volunteer has many cancellations.
- Volunteer has many successful events.
- Historical records are corrected.
- A volunteer's reliability calculation changes after attendance correction.

Never present insufficient data as a highly accurate reliability score.

---

### **NGO Edge Cases**

- NGO creates an event with incomplete requirements.
- NGO changes event requirements after volunteers register.
- NGO account is disabled.
- NGO attempts to access another NGO's event.
- NGO tries to modify historical attendance.
- NGO submits transparency information that is not platform-verified.
- NGO leaves the platform while events are active.

---

### Emergency Edge Cases

- Emergency is accidentally triggered twice.
- Emergency is created by an unauthorized user.
- No eligible volunteers exist.
- Volunteers receive the alert but nobody confirms.
- A volunteer accepts and then withdraws.
- Emergency requirements change after broadcasting.
- Firebase notification service fails.
- Email fallback fails.
- NGO tries to deploy without enough confirmed volunteers.
- Agent attempts to initiate an emergency.

The last case must always be rejected.

---

### Agent Edge Cases

- Agent service is unavailable.
- Agent returns an invalid response.
- Agent generates unsupported claims.
- Agent receives incomplete event information.
- Agent attempts to modify authoritative data.
- Agent receives information belonging to another NGO.
- Agent stores sensitive information unnecessarily.
- Agent response takes too long.
- User repeatedly requests expensive Agent operations.

Agent requests should have authentication, authorization, validation, logging, and rate limits.

---

## 37. Privacy Rules

SevaSetu should follow a simple principle:

If the platform does not need the data, don't collect it.

Particularly protect:

- Phone numbers

- Email addresses
- Exact volunteer locations
- Availability information
- Internal NGO information
- Attendance records
- Reliability history
- Private Agent context

Public profiles should expose only information necessary for community discovery.

---

### **38. Location Privacy**

Exact volunteer location should not be publicly exposed.

The platform can use location internally for matching.

For example:

Internally:

Volunteer coordinates

+

Event coordinates

=

Distance

Publicly:

"Approximately 4 km away"

or simply:

"Nearby"

when appropriate.

---

### **39. API Security**

Every protected API should:

1. Verify authentication.
2. Verify the user's role.
3. Verify ownership/access.
4. Validate input.

5. Perform the operation.
6. Return a safe response.
7. Log sensitive actions where necessary.

Example:

POST /events/:id/attendance

should verify:

Is user logged in?

↓

Is user an authorized NGO member?

↓

Does event belong to this NGO?

↓

Is attendance operation valid?

↓

Save attendance

↓

Create audit record

---

## 40. Rate Limiting

Rate limits should be applied to:

- Login-related APIs
- Agent requests
- Event creation
- Registration attempts
- Emergency operations
- Notification triggers
- Public search APIs

This reduces abuse and accidental overload.

---

## 41. Secrets Management

Never place the following in frontend code:

- Supabase service-role key
- Firebase server credentials
- Clerk secret keys
- AI API keys
- Email API keys
- Maps API private keys

Use environment variables and server-side secrets.

The Supabase service-role key must never be exposed to the browser.

---

## **42. Audit Logging**

At minimum, log:

- Login/security events where appropriate
- Role changes
- Event creation
- Event deletion/cancellation
- Roster confirmation
- Attendance creation
- Attendance correction
- Karma creation/correction
- Emergency creation
- Emergency deployment decisions
- Administrative actions

Each audit record should contain:

- Who performed the action
  - What happened
  - Which record was affected
  - When it happened
- 

## **43. Security Launch Checklist**

Before the hackathon/demo or public launch, verify:

### **Authentication**

- Clerk authentication works.
- Sessions expire correctly.
- Protected routes require authentication.
- Role checks work.

### **Database**

- Supabase RLS is enabled.
- Volunteers cannot access other volunteers' private data.
- NGOs cannot access other NGOs' private data.
- Public data is intentionally exposed.
- Service-role credentials never reach the frontend.

### **Karma**

- Volunteers cannot award themselves Karma.
- Duplicate Karma transactions are prevented.
- Karma comes from verified participation.
- Karma is separate from reliability.

### **Attendance**

- Volunteers cannot modify attendance.
- NGO authorization is checked.
- Duplicate attendance is prevented.
- Corrections are logged.

### **Matching**

- Mandatory requirements are enforced.
- Matching is deterministic.
- Match explanations use actual data.
- Karma is not used as a direct matching score.

### **Agent**

- Agent has restricted access.
- Agent cannot modify authoritative records.
- Agent cannot trigger emergencies.
- Agent failures do not break the platform.
- Agent output is treated as recommendations.

## **Emergency**

- Only authorized NGO users can initiate emergencies.
- Volunteers must explicitly confirm.
- Emergency deployment requires human decision.
- Duplicate emergency alerts are prevented.

## **Privacy**

- Exact volunteer locations are not publicly exposed.
- Private availability is protected.
- Private NGO information is protected.
- Agent context is protected.

## **Error Handling**

- Database failures are handled.
- Agent failures are handled.
- Maps failures are handled.
- Notification failures are handled.
- Authentication failures are handled.
- No sensitive technical errors are shown to users.

## **Deployment**

- Secrets are stored securely.
- Production environment variables are configured.
- CORS is restricted appropriately.
- HTTPS is used.
- Production database backups/recovery strategy is available.
- Audit logs are protected.

---

## **44. Security Priority for the Hackathon**

Do not spend equal effort on every security feature.

For the SevaSetu MVP, the highest priorities are:

1. Authentication

↓

2. Role-based authorization

↓

3. Supabase RLS

↓

4. Protect attendance

↓

5. Protect Karma

↓

6. Protect NGO/event ownership

↓

7. Restrict Agent permissions

↓

8. Protect emergency actions

↓

9. Error handling

↓

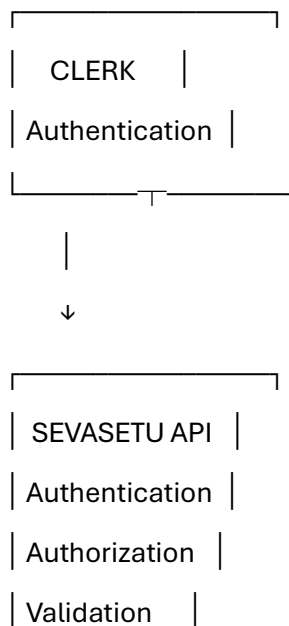
10. Audit logging

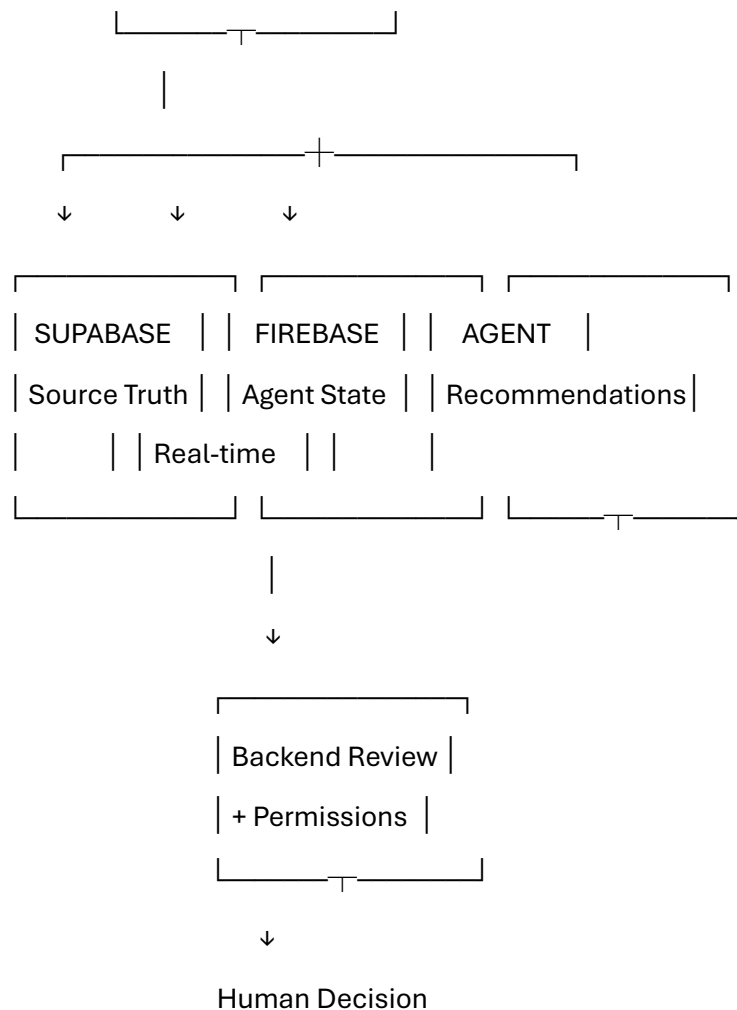
Advanced security features can come later.

---

## 45. Final Security Architecture

The recommended security model is:





The most important architectural rule is:

**The Agent can provide intelligence, but the backend/database remains in control of authoritative actions.**

That principle should remain true even as SevaSetu grows.

#### 46. Security Definition of Done

SevaSetu is security-ready for its MVP when:

1. Every user is authenticated before accessing protected functionality.
2. Every protected action checks the user's role and ownership.
3. Supabase RLS prevents unauthorized database access.
4. Volunteers cannot manipulate attendance, Karma, or reliability.
5. NGOs can only manage their own organization and events.
6. Public users see only intentionally public information.
7. The Agent has restricted permissions.
8. The Agent cannot perform irreversible actions.

9. Emergency actions require human authorization.
10. Core platform operations continue even if the Agent fails.
11. Sensitive errors are logged internally but not exposed to users.
12. Critical actions have an audit trail.
13. Secrets never reach the frontend.
14. Major edge cases have automated or manual tests.
15. The complete volunteer journey can be executed safely using test data.

**Core principle:**

**SevaSetu should fail safely rather than pretend that an operation succeeded.**